

18 cars passing a certain tollboth at different time of the day (x) with different speed (y)

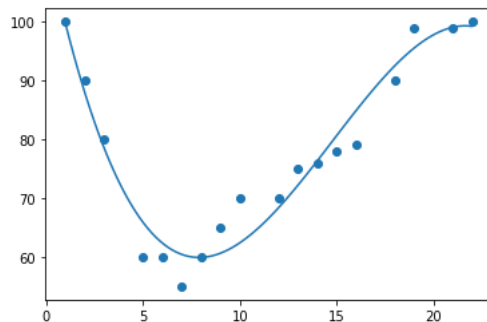
```
import numpy
import matplotlib.pyplot as plt

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

#NumPy has a method that lets us make a polynomial model
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

#specify how the line will display, we start at position 1, and end at position 22
myline = numpy.linspace(1, 22, 100)

plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()
```



It is important to know how well the relationship between the values of the x- and y-axis is, if there are no relationship the polynomial regression can not be used to predict anything.

The relationship is measured with a value called the r-squared.

The r-squared value ranges from 0 to 1, where 0 means no relationship, and 1 means 100% related

```
import numpy
from sklearn.metrics import r2_score

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

print(r2_score(y, mymodel(x)))

0.9432150416451027
```

✓ Predict Future Values

Let us try to predict the speed of a car that passes the tollbooth at around 17 P.M

```
import numpy
from sklearn.metrics import r2_score

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

speed = mymodel(17)
print(speed)

88.87331269697987
```

img_polynomial_prediction.png

✓ Amendments

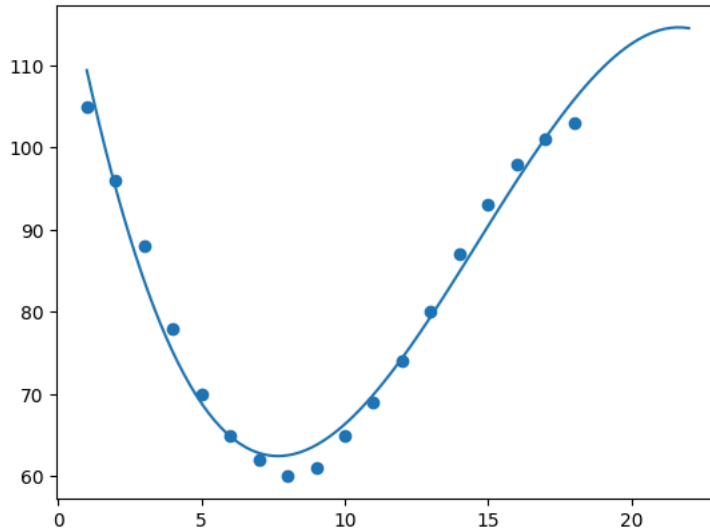
```
import numpy
import matplotlib.pyplot as plt

x = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18]
y = [105,96,88,78,70,65,62,60,61,65,69,74,80,87,93,98,101,103]

#NumPy has a method that lets us make a polynomial model
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

#specify how the line will display, we start at position 1, and end at position 22
myline = numpy.linspace(1, 22, 100)

plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()
```



```
import numpy
from sklearn.metrics import r2_score

x = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18]
y = [105,96,88,78,70,65,62,60,61,65,69,74,80,87,93,98,101,103]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

print(r2_score(y, mymodel(x)))
```

```
0.9791401626339165
```

```
import numpy
from sklearn.metrics import r2_score

x = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18]
y = [105,96,88,78,70,65,62,60,61,65,69,74,80,87,93,98,101,103]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

speed = mymodel(19)
print(speed)
```

```
109.71568627450986
```

✓ Conclusion

I modified the dataset and applied a third degree polynomial regression model to fit a nonlinear relationship between the variables. After changing the data, the fitted curve adjusted to reflect the new pattern, whilst the R2 score remained high, indicating that the model continued to explain most of the variation in the data. I also changed the prediction point from 17 to 19 and observed a different prediction value. This demonstrates that polynomial regression is useful for modelling curved relationships that cannot be accurately represented using a straight line regression model.

Start coding or [generate](#) with AI.

